Руководство по эксплуатации

Документ описывает функциональные характеристики программы для ЭВМ «XML-библиотека „Берёста“ (beresta)» и порядок работы с ней: программный интерфейс на C++ и плоский Си-интерфейс, примеры разбора (DOM и SAX), запросов XPath и проверки по XML Schema, обработку ошибок и безопасность. Установка и сборка описаны в «Руководстве по установке и развёртыванию».

Правообладатель: [ФИО правообладателя].

Содержание

1. Функциональные характеристики
2. Порядок работы
3. Работа на C++
4. Работа через Си-интерфейс
5. Справочник Си-интерфейса
6. Обёртки для других языков
7. Обработка ошибок
8. Безопасность
9. Потокобезопасность
10. Ограничения

Функциональные характеристики

«Берёста» — библиотека, которую встраивают в прикладные программы для работы с документами XML. Пользовательского интерфейса у неё нет: функции вызываются из кода программы. Все сообщения об ошибках выводятся на русском языке (по умолчанию) или на английском.

Возможность	Стандарт и объём поддержки
Разбор XML	XML 1.0 (5-я редакция): 100 % применимых случаев W3C XML Conformance Test Suite
Пространства имён	Namespaces in XML: проверка при разборе, URI вычисляются по текущему месту узла в дереве
Кодировки	UTF-8, UTF-16 LE/BE, ISO-8859-1, US-ASCII, windows-1251, windows-1252, KOI8-R, IBM866 (собственные таблицы)
Дерево документа (DOM)	создание, навигация, изменение, копирование и перенос узлов между документами
Потоковый разбор (SAX)	события без построения дерева, память не зависит от размера документа

Возможность	Стандарт и объём поддержки
DTD	внутреннее и внешнее подмножества, параметрические сущности, атрибуты по умолчанию, проверка валидности
XML Schema	XSD 1.0: 45 встроенных типов, фасеты, наследование, группы замены, `xs:key` / `xs:unique` / `xs:keyref`, `include` / `import` / `redefine`; из XSD 1.1 — `xs:assert`, `xs:assertion`, `xs:override`. W3C XML Schema Test Suite — 96,2 % применимых случаев
XPath	XPath 1.0 полностью: 13 осей, 27 функций, переменные и префиксы
XInclude	XInclude 1.0: `parse="xml|text"`, `xi:fallback`, XPointer `element()` и по ID
Обход дерева	DOM Level 2 Traversal: `TreeWalker`, `NodeIterator` с фильтрами
Запись XML	компактный и форматированный вывод, любая поддерживаемая выходная кодировка
Другие языки	Си-интерфейс (71 функция) и обёртки Java 22+, C#, Python, Node.js, WebAssembly

Сознательные ограничения перечислены в разделе «Ограничения» в конце документа.

Порядок работы

1. Подключите библиотеку к проекту (см. «Руководство по установке и развёртыванию»).
2. Программа на C++ подключает один заголовок `beresta.hpp`; программа на C или другом языке — `capi/beresta_c.h` или пакет своего языка.
3. Разберите документ (`XmlParser`, `beresta_document_parse`) или создайте его с нуля.
4. Работайте с деревом, вычисляйте XPath, проверяйте по схеме.
5. Запишите результат в строку или файл.
6. Обрабатывайте ошибки: в C++ — исключения `beresta::DomException`, в C — коды состояния и `beresta_last_error_message()`.

Работа на C++

Разбор, поиск, изменение, сохранение (DOM)

```
#include "beresta.hpp"
#include <iostream>

int main()
{
    try
    {
        beresta::Document doc = beresta::XmlParser().parseFile("library.xml");

        for (beresta::Element* book : beresta::xpath::selectElements(doc, "//book[@lang='ru']"))
            book->setAttribute("checked", "yes");

        beresta::SerializerOptions options;
        options.prettyPrint = true;
        beresta::XmlSerializer(options).writeFile(doc, "library.out.xml");
    }
    catch (const beresta::DomException& error)
    {
        std::cerr << error.what() << '\n';     // [PARSE_ERROR] строка 3, столбец 7: ...
        return 1;
    }
    return 0;
}
```

Построение документа с нуля

```
beresta::Document doc;
doc.setDeclaration(beresta::XmlDeclaration{"1.0", "UTF-8", std::nullopt});

beresta::Element* root = doc.setRoot(std::make_unique<beresta::Element>("note"));
root->setAttribute("title", "Q&A");
root->appendChild(std::make_unique<beresta::Comment>(" черновик "));

auto body = std::make_unique<beresta::Element>("body");
body->setTextContent("1 < 2");
root->appendChild(std::move(body));

std::cout << doc.toString();
// <?xml version="1.0" encoding="UTF-8"?><note title="Q&amp;A"><!-- черновик --><body>1 &lt;
2</body></note>
```

Родитель владеет детьми (`std::unique_ptr`), поэтому освобождать узлы вручную не нужно. Основные настройки разбора — структура `ParserOptions`: `preserveWhitespaceText`, `discardComments`, `trackLocations`, `validateDtd`, `processXInclude`, лимиты `maxDepth` (1024), `maxEntityExpansions` (10 000), `maxExpandedSize` (10 МБ), `resolver`, `errorHandler`, `schema`.

Потоковый разбор (SAX)

Дерево не строится, расход памяти не зависит от размера документа:

```
class PriceCounter : public beresta::ContentHandler
{
public:
    void startElement(const beresta::SaxElement& e) override { inside = e.localName == "price"; }
    void characters(std::string_view text) override { if (inside) total +=
beresta::xpath::stringToNumber(text); }
    void endElement(const beresta::SaxElement&) override { inside = false; }

    bool inside = false;
    double total = 0;
};

PriceCounter counter;
beresta::SaxParser(options).parseFile("big.xml", counter);
```

События: `startDocument` / `endDocument`, `xmlDeclaration`, `documentType`, `startElement` / `endElement`, `characters`, `cdata`, `ignorableWhitespace`, `comment`,

`processingInstruction`. Строки `string_view` в событиях действительны только на время вызова.

Запросы XPath 1.0

```
// Быстрые функции
std::vector<beresta::Element*> books = beresta::xpath::selectElements(doc, "//book[price >
300]");
std::string title = beresta::xpath::evaluateString(doc, "string(//book[1]/title)");

// Скомпилированное выражение: многократно и из разных потоков
beresta::xpath::XPathExpression expression("//s:item[@code = $code]/s:name");
beresta::xpath::XPathContext context;
context.namespaces.emplace("s", "urn:shop");
context.variables.emplace("code", beresta::xpath::XPathValue::fromString("B2"));

beresta::xpath::XPathValue value = expression.evaluate(doc, context);
value.getType();          // NODE_SET, BOOLEAN, NUMBER, STRING
```

Проверка по XML Schema

```
// Загрузка схемы (один раз; схема неизменяема и разделяется между потоками)
std::shared_ptr<const beresta::xsd::Schema> schema =
beresta::xsd::Schema::fromFile("catalog.xsd", loadOptions);

// Вариант 1: проверка готового дерева
beresta::xsd::SchemaValidator validator(schema);
std::vector<beresta::Diagnostic> errors = validator.validate(doc);

// Вариант 2: проверка при разборе в DOM — один проход
beresta::ParserOptions options;
options.schema = schema;
options.errorHandler = &handler;          // без него — исключение на первой ошибке
beresta::XmlParser(options).parseFile("catalog.xml");

// Вариант 3: потоковая проверка без дерева — для выгрузок в гигабайты
MyHandler events;                         // любой ContentHandler
beresta::SaxParser(options).parseFile("export.xml", events);
```

Каждое нарушение содержит позицию и путь к элементу, например `строка 12, столбец 5: /catalog/book[2]/price: значение "-5" меньше минимума 0`. Во всех трёх вариантах нарушения, их порядок и позиции совпадают.

Работа через Си-интерфейс

Заголовок `include/capi/beresta_c.h` написан на C99 и подключается и в `.c`, и в `.cpp`. На нём построены обёртки Java, C#, Python и Node.js; им же пользуются из C, Go (cgo), Rust (FFI), Delphi и любого языка, умеющего вызывать функции C.

Соглашения

Правило	Пояснение
Дескрипторы	`beresta_document`, `beresta_node`, `beresta_xpath`, `beresta_nodeset`, `beresta_schema`, `beresta_diagnostics` — непрозрачные указатели `void*`
Статус	функции, возвращающие `int`, отдают `BERESTA_OK` (0) при успехе или код ошибки
NULL	функции, возвращающие дескриптор или `char*`, отдают NULL при ошибке; NULL с кодом 0 означает «объекта нет»
Последняя ошибка	`beresta_last_error_code()` и `beresta_last_error_message()` — своя у каждого потока; каждый вызов её сбрасывает
Строки	UTF-8; `char*` от библиотеки освобождается через `beresta_string_free`, `const char*` — заимствованная
Размеры и индексы	`int64_t`; `-1` в результате означает ошибку
Владение	документ, схему, выражение, набор узлов и список нарушений освобождает вызывающий (`beresta_*_free`); узел принадлежит документу
Потоки	разные документы — в разных потоках параллельно; схема и скомпилированное выражение потокобезопасны

Пример: DOM и XPath на C

```
#include "capi/beresta_c.h"
#include <stdio.h>
#include <string.h>

const char* xml = "<shop><item code=\"A1\">Чай</item><item code=\"B2\">Кофе</item></shop>";
beresta_document document = NULL;
beresta_xpath xpath = NULL;

if (beresta_document_parse(xml, (int64_t)strlen(xml), BERESTA_PARSE_DEFAULT, NULL, &document) !=
BERESTA_OK)
{
    fprintf(stderr, "[%s] %s\n", beresta_status_name(beresta_last_error_code()),
beresta_last_error_message());
    return 1;
}
beresta_xpath_compile("//item[@code = $code]", &xpath);
beresta_xpath_set_string(xpath, "code", "B2");

char* name = beresta_xpath_string(xpath, document);   /* "Кофе" */
printf("%s\n", name);
beresta_string_free(name);                             /* строку освобождает библиотека */

beresta_xpath_free(xpath);
beresta_document_free(document);                       /* документ вместе со всеми узлами */
```

Пример: построение документа на C

```
beresta_document document = NULL;
beresta_node item;

if (beresta_document_create("order", &document) != BERESTA_OK)
    return 1;
item = beresta_node_append_element(beresta_document_root(document), "item");
beresta_element_set_attribute(item, "qty", "2");
beresta_node_append_text(item, "Мёд & орехи");

char* text = beresta_node_to_string(document, BERESTA_WRITE_PRETTY);
printf("%s\n", text);
beresta_string_free(text);
beresta_document_free(document);
```

Пример: потоковая проверка файла по схеме на C

```
beresta_schema schema = NULL;
beresta_diagnostics problems = NULL;

if (beresta_schema_load_file("catalog.xsd", &schema) != BERESTA_OK)
    return 1;
if (beresta_schema_validate_file("export.xml", BERESTA_PARSE_DEFAULT, schema, &problems) ==
BERESTA_OK)
{
    for (int64_t i = 0; i < beresta_diagnostics_count(problems); ++i)
    {
        char* text = beresta_diagnostics_message(problems, i);
        printf("%lld: %s\n", (long long)beresta_diagnostics_line(problems, i), text);
        beresta_string_free(text);
    }
    beresta_diagnostics_free(problems);
}
beresta_schema_free(schema);
```

Полный пример на C — файл `examples/capi/capi_demo.c` (собирается вместе с библиотекой как `beresta_capi_demo`).

Справочник Си-интерфейса

Справочник построен из заголовка `include/capi/beresta_c.h` (функций: 71).

Коды состояния

Имя	Значение	Описание
`BERESTA_OK`	`0`	успех
`BERESTA_INVALID_NAME`	`1`	недопустимое XML-имя
`BERESTA_INVALID_CONTENT`	`2`	недопустимое содержимое узла
`BERESTA_HIERARCHY_VIOLATION`	`3`	нарушение структуры дерева
`BERESTA_INDEX_OUT_OF_RANGE`	`4`	индекс вне диапазона
`BERESTA_NULL_NODE`	`5`	пустой узел внутри ядра
`BERESTA_PARSE_ERROR`	`6`	синтаксическая ошибка XML
`BERESTA_NOT_FOUND`	`7`	объект не найден
`BERESTA_NAMESPACE_ERROR`	`8`	нарушение правил пространств имён
`BERESTA_IO_ERROR`	`9`	ошибка чтения или записи файла

Имя	Значение	Описание
`BERESTA_ENCODING_ERROR`	`10`	неизвестная или неверная кодировка
`BERESTA_VALIDATION_ERROR`	`11`	документ не соответствует DTD / XSD
`BERESTA_XPATH_ERROR`	`12`	ошибка XPath
`BERESTA_SCHEMA_ERROR`	`13`	некорректная XML Schema
`BERESTA_RESOURCE_ERROR`	`14`	внешний ресурс не загружен
`BERESTA_LIMIT_EXCEEDED`	`15`	превышен защитный лимит
`BERESTA_INVALID_ARGUMENT`	`16`	NULL или дескриптор не того типа
`BERESTA_OUT_OF_MEMORY`	`17`	не хватило памяти
`BERESTA_UNKNOWN_ERROR`	`18`	непредвиденное исключение

Флаги разбора и вывода

Имя	Значение	Описание
`BERESTA_PARSE_DEFAULT`	`0u`	настройки по умолчанию
`BERESTA_PARSE_PRESERVE_WHITESPACE`	`(1u << 0)`	хранить пробельный текст между тегами
`BERESTA_PARSE_DISCARD_COMMENTS`	`(1u << 1)`	не создавать узлы комментариев
`BERESTA_PARSE_COALESCE_CDATA`	`(1u << 2)`	CDATA превращать в обычный текст
`BERESTA_PARSE_VALIDATE_DTD`	`(1u << 3)`	проверять документ по DTD
`BERESTA_PARSE_NO_NAMESPACES`	`(1u << 4)`	отключить обработку xmlns
`BERESTA_PARSE_XINCLUDE`	`(1u << 5)`	выполнять xi:include
`BERESTA_PARSE_FILE_RESOURCES`	`(1u << 6)`	разрешить внешние DTD/сущности/XInclude из каталога разбираемого файла
`BERESTA_PARSE_TRACK_LOCATIONS`	`(1u << 7)`	запоминать строку и столбец узлов: тогда beresta_schema_validate даёт места нарушений (разбор ~8 % медленнее; потоковой проверке флаг не нужен — места известны и так)

Имя	Значение	Описание
`BERESTA_WRITE_DEFAULT`	`0u`	компактно, без лишних пробелов
`BERESTA_WRITE_PRETTY`	`(1u << 0)`	отступы и переводы строк
`BERESTA_WRITE_DECLARATION`	`(1u << 1)`	всегда писать <?xml ...?>

Библиотека и ошибки

- `const char* beresta_version(void)` — Версия библиотеки, например "2.1.0" (заимствованная строка)
- `int beresta_last_error_code(void)` — Код последней ошибки текущего потока (BERESTA_OK, если ошибки не было)
- `const char* beresta_last_error_message(void)` — Сообщение последней ошибки текущего потока ("" — ошибки не было). Заимствованная строка: действительна до следующего вызова в этом потоке
- `const char* beresta_status_name(int status)` — Имя кода состояния, например "PARSE_ERROR" (заимствованная строка)
- `void beresta_string_free(char* text)` — Освобождение строки, которую вернула библиотека (NULL допустим)

Языки сообщений об ошибках

- `int beresta_set_language(const char* code)` — Смена языка сообщений для всех потоков ("ru", "en", ...). BERESTA_INVALID_ARGUMENT — такого языка нет, язык не меняется.
- `const char* beresta_language(void)` — Код текущего языка (заимствованная строка)
- `const char* beresta_languages(void)` — Все языки через запятую, например "ru,en" (заимствованная строка)
- `const char* beresta_message_text(int message)` — Текст сообщения на текущем языке (заимствованная строка; {0} — место аргумента)

Документы

- `int beresta_document_create(const char* root_name, beresta_document* out)` — Новый документ; root_name == NULL — документ без корневого элемента
- `int beresta_document_parse(const void* data, int64_t size, uint32_t flags, beresta_schema schema, beresta_document* out)` — Разбор байтов в любой поддерживаемой кодировке (UTF-8/16, windows-1251...). schema == NULL — без проверки по схеме
- `int beresta_document_parse_file(const char* path, uint32_t flags, beresta_schema schema, beresta_document* out)` — Разбор файла (кодировка определяется автоматически)

- `void beresta_document_free(beresta_document document)` — Освобождение документа вместе со всеми его узлами (NULL допустим)
- `int beresta_document_clone(beresta_document document, beresta_document* out)` — Глубокая копия документа — новый независимый документ
- `beresta_node beresta_document_root(beresta_document document)` — Корневой элемент или NULL, если его нет
- `beresta_node beresta_document_element_by_id(beresta_document document, const char* id)` — Элемент с атрибутом типа ID (по DTD) или с атрибутом xml:id
- `int beresta_document_save(beresta_document document, const char* path, uint32_t flags, const char* encoding)` — Запись документа в файл; encoding == NULL — UTF-8

Узлы: свойства и навигация

- `int beresta_node_type(beresta_node node)` — Тип узла (значение beresta_node_kind) или -1 при ошибке
- `char* beresta_node_name(beresta_node node)` — Имя: тег элемента, цель PI, имя DOCTYPE; "#text", "#comment" и т.п.
- `char* beresta_node_text(beresta_node node)` — Текстовое содержимое узла (у элемента — весь текст потомков)
- `char* beresta_node_to_string(beresta_node node, uint32_t flags)` — Сериализация узла и его потомков в строку UTF-8
- `beresta_node beresta_node_parent(beresta_node node)` — Родительский узел или NULL у корня дерева
- `beresta_node beresta_node_first_child(beresta_node node)` — Первый дочерний узел или NULL
- `beresta_node beresta_node_last_child(beresta_node node)` — Последний дочерний узел или NULL
- `beresta_node beresta_node_next_sibling(beresta_node node)` — Следующий соседний узел или NULL
- `beresta_node beresta_node_previous_sibling(beresta_node node)` — Предыдущий соседний узел или NULL
- `beresta_document beresta_node_owner_document(beresta_node node)` — Документ, которому принадлежит узел (у документа — он сам)
- `int64_t beresta_node_child_count(beresta_node node)` — Число дочерних узлов (0 у листовых узлов) или -1 при ошибке
- `beresta_node beresta_node_child(beresta_node node, int64_t index)` — Дочерний узел по индексу

Узлы: изменение дерева

- `int beresta_node_set_text(beresta_node node, const char* text)` — Замена текста: у элемента — все дети заменяются одним текстовым узлом, у текста / комментария / CDATA / PI — новое значение

- `int beresta_node_remove(beresta_node node)` — Удаление узла из дерева вместе с потомками; дескриптор становится недействительным
- `int beresta_node_copy_to(beresta_node source, beresta_node parent, beresta_node* out)` — Глубокая копия source в конец детей parent (можно между документами)
- `int beresta_node_move_to(beresta_node node, beresta_node parent)` — Перенос узла (без копирования) в конец детей parent, в том числе в другой документ. Дескрипторы узла и его потомков остаются действительными и теперь принадлежат документу parent: освобождение прежнего документа их не затрагивает. Ошибки (дерево при этом не меняется): BERESTA_INVALID_ARGUMENT — NULL, parent не элемент и не документ, node — сам документ; BERESTA_HIERARCHY_VIOLATION — node — корневой элемент документа, parent — сам node или его потомок, или parent не принимает узел такого типа (текст в документ).
- `beresta_node beresta_node_append_element(beresta_node parent, const char* name)` — Добавление нового узла в конец детей parent (элемента или документа). У документа append_element задаёт корневой элемент
- `beresta_node beresta_node_append_text(beresta_node parent, const char* text)` — Добавление нового узла в конец детей parent (элемента или документа). У документа append_element задаёт корневой элемент
- `beresta_node beresta_node_append_comment(beresta_node parent, const char* text)` — Добавление нового узла в конец детей parent (элемента или документа). У документа append_element задаёт корневой элемент
- `beresta_node beresta_node_append_cdata(beresta_node parent, const char* text)` — Добавление нового узла в конец детей parent (элемента или документа). У документа append_element задаёт корневой элемент
- `beresta_node beresta_node_append_pi(beresta_node parent, const char* target, const char* data)` — Добавление нового узла в конец детей parent (элемента или документа). У документа append_element задаёт корневой элемент

Элементы и атрибуты

- `char* beresta_element_get_attribute(beresta_node element, const char* name)` — Значение атрибута или NULL (код 0), если атрибута нет
- `int beresta_element_set_attribute(beresta_node element, const char* name, const char* value)` — Добавление или замена атрибута (позиция прежнего атрибута сохраняется)
- `int beresta_element_has_attribute(beresta_node element, const char* name)` — 1 — атрибут есть, 0 — нет, -1 — ошибка
- `int beresta_element_remove_attribute(beresta_node element, const char* name)` — 1 — атрибут удалён, 0 — его не было, -1 — ошибка
- `int64_t beresta_element_attribute_count(beresta_node element)` — Перебор атрибутов в порядке документа
- `char* beresta_element_attribute_name(beresta_node element, int64_t index)` — Перебор атрибутов в порядке документа

- `char* beresta_element_attribute_value(beresta_node element, int64_t index)` — Перебор атрибутов в порядке документа
- `char* beresta_element_local_name(beresta_node element)` — Локальное имя (без префикса)
- `char* beresta_element_namespace_uri(beresta_node element)` — URI пространства имён или NULL (код 0), если элемент вне пространств имён

XPath 1.0

- `int beresta_xpath_compile(const char* expression, beresta_xpath* out)` — Компиляция выражения (один раз — вычислять можно многократно)
- `void beresta_xpath_free(beresta_xpath xpath)` — Освобождение скомпилированного выражения (NULL допустим)
- `int beresta_xpath_bind_namespace(beresta_xpath xpath, const char* prefix, const char* uri)` — Привязка префикса к URI для имён вида p:name в выражении
- `int beresta_xpath_set_string(beresta_xpath xpath, const char* name, const char* value)` — Значения переменных $name
- `int beresta_xpath_set_number(beresta_xpath xpath, const char* name, double value)` — Значения переменных $name
- `int beresta_xpath_select(beresta_xpath xpath, beresta_node context, beresta_nodeset* out)` — Вычисление: множество узлов / строка / число / логическое (1, 0, -1 — ошибка)
- `char* beresta_xpath_string(beresta_xpath xpath, beresta_node context)` — Вычисление: множество узлов / строка / число / логическое (1, 0, -1 — ошибка)
- `int beresta_xpath_number(beresta_xpath xpath, beresta_node context, double* out)` — Вычисление: множество узлов / строка / число / логическое (1, 0, -1 — ошибка)
- `int beresta_xpath_boolean(beresta_xpath xpath, beresta_node context)` — Вычисление: множество узлов / строка / число / логическое (1, 0, -1 — ошибка)
- `int64_t beresta_nodeset_size(beresta_nodeset nodeset)` — Результат select: размер, узел (NULL для атрибута), строковое значение
- `beresta_node beresta_nodeset_node(beresta_nodeset nodeset, int64_t index)` — Результат select: размер, узел (NULL для атрибута), строковое значение
- `char* beresta_nodeset_value(beresta_nodeset nodeset, int64_t index)` — Результат select: размер, узел (NULL для атрибута), строковое значение
- `void beresta_nodeset_free(beresta_nodeset nodeset)` — Освобождение набора узлов (NULL допустим)

XML Schema

- `int beresta_schema_load(const void* data, int64_t size, beresta_schema* out)` — Загрузка схемы из байтов XSD (xs:include / xs:import недоступны)
- `int beresta_schema_load_file(const char* path, beresta_schema* out)` — Загрузка из файла: xs:include / xs:import ищутся в каталоге этого файла

- `void beresta_schema_free(beresta_schema schema)` — Освобождение схемы (NULL допустим)
- `int beresta_schema_validate(beresta_schema schema, beresta_document document, beresta_diagnostics* out)` — Проверка документа: все нарушения собираются за один проход в out
- `int beresta_schema_validate_bytes(const void* data, int64_t size, uint32_t flags, beresta_schema schema, beresta_diagnostics* out)` — Потоковая проверка без дерева (SAX + XSD): аргументы те же, что у beresta_document_parse / beresta_document_parse_file, но вместо документа в out — список нарушений. Память — O(глубины документа) плюс значения ID и таблицы ключей; файл отображается в память, а не копируется, поэтому так проверяются выгрузки в гигабайты. Нарушение схемы — не ошибка вызова (BERESTA_OK, нарушения в out); синтаксическая ошибка XML — BERESTA_PARSE_ERROR.
- `int beresta_schema_validate_file(const char* path, uint32_t flags, beresta_schema schema, beresta_diagnostics* out)` — Потоковая проверка файла без построения дерева (см. beresta_schema_validate_bytes)
- `int64_t beresta_diagnostics_count(beresta_diagnostics diagnostics)` — Список нарушений: число, текст, строка и столбец в исходнике (0 — неизвестно)
- `char* beresta_diagnostics_message(beresta_diagnostics diagnostics, int64_t index)` — Список нарушений: число, текст, строка и столбец в исходнике (0 — неизвестно)
- `int64_t beresta_diagnostics_line(beresta_diagnostics diagnostics, int64_t index)` — Список нарушений: число, текст, строка и столбец в исходнике (0 — неизвестно)
- `int64_t beresta_diagnostics_column(beresta_diagnostics diagnostics, int64_t index)` — Список нарушений: число, текст, строка и столбец в исходнике (0 — неизвестно)
- `void beresta_diagnostics_free(beresta_diagnostics diagnostics)` — Освобождение списка нарушений (NULL допустим)

Обёртки для других языков

Язык	Установка	Основа
Java 22+	Maven `io.github.ermaleyalekov:beresta`	Project Panama (Foreign Function & Memory API), без JNI
C#	`dotnet add package Beresta`	P/Invoke и `SafeHandle`
Python 3.8+	`pip install beresta`	ctypes, без компилятора на машине пользователя
Node.js 18+	`npm install beresta`	собственный модуль Node-API, типы TypeScript в комплекте
Браузер, Workers	`npm install beresta-wasm`	WebAssembly (Emscripten), тот же JS-слой

Обёртки повторяют возможности Си-интерфейса: DOM, XPath, XML Schema, включая потоковую проверку без дерева. Подробно — в файле `docs/bindings.md` репозитория.

Обработка ошибок

В C++ все ошибки — исключения `beresta::DomException` (код и сообщение) и производное `beresta::ParseException` (с позицией: смещение, строка, столбец). Коды ошибок `DomErrorCode`: `PARSE_ERROR`, `NAMESPACE_ERROR`, `ENCODING_ERROR`, `VALIDATION_ERROR`, `RESOURCE_ERROR`, `LIMIT_EXCEEDED`, `SCHEMA_ERROR`, `XPATH_ERROR`, `INVALID_NAME`, `INVALID_CONTENT`, `HIERARCHY_VIOLATION`, `INDEX_OUT_OF_RANGE`, `NULL_NODE`, `NOT_FOUND`, `IO_ERROR`.

```
try { /* ... */ }
catch (const beresta::ParseException& e) { e.getLine(); e.getColumn(); e.getCode(); }
catch (const beresta::DomException& e)   { e.getCode(); e.getMessage();
beresta::errorCodeToString(e.getCode()); }
```

Нефатальные ошибки (нарушения DTD и XSD) можно собирать обработчиком `ErrorHandler` (готовый — `CollectingErrorHandler`), не прерывая разбор. В Си-интерфейсе исключения не пересекают границу библиотеки: каждая функция возвращает код состояния (таблица «Коды состояния» выше), а текст — `beresta_last_error_message()`.

Язык сообщений

Язык по умолчанию задаётся при сборке (`make ERRORS_LANG=en`, `-DBERESTA_ERRORS_LANG=en`; без параметра — русский) и меняется во время работы одним вызовом для всех потоков: `beresta::i18n::setLanguage("en")` в C++, `beresta_set_language("en")` в C. Имя кода ошибки одинаково во всех языках:

```
[PARSE_ERROR] строка 1, столбец 7: ожидался </b>, найден </a>
[PARSE_ERROR] line 1, column 7: expected </b>, found </a>
```

Безопасность

- По умолчанию **ничего внешнего не загружается**: ни внешний DTD, ни внешние сущности, ни XInclude, ни `xs:include`. Это закрывает атаки XXE и SSRF. Загрузку разрешает только явно переданный загрузчик (`FileResourceResolver` — только локальные файлы внутри заданного каталога, без сетевых адресов) или флаг `BERESTA_PARSE_FILE_RESOURCES` в Си-интерфейсе.
- Лимиты `maxEntityExpansions`, `maxEntityDepth`, `maxExpandedSize` и `maxDepth` всегда включены и защищают от «сущностных бомб» и слишком глубоких документов; превышение — ошибка `LIMIT_EXCEEDED`.
- Регулярные выражения XSD выполняются за линейное время; хеш-таблицы с ключами из документа защищены от подбора коллизий (SipHash со случайным ключом процесса).
- Рекурсии по дереву нет: разбор, копирование, вывод, проверка и удаление дерева обходят его по явному стеку.

Потокобезопасность

- `XmlParser`, `SaxParser`, `XmlSerializer`, `XPathExpression`, `Schema`, `SchemaValidator` неизменяемы: один объект можно использовать из многих потоков одновременно.
- Разные деревья обрабатываются параллельно без синхронизации; одно дерево можно читать из многих потоков, а при изменении нужна внешняя синхронизация.

Ограничения

- XML Schema: при загрузке схемы не проверяются корректность ограничения модели содержимого и однозначность модели (UPA); документ схемы проверяется по схеме для схем не целиком; из XSD 1.1 поддерживаются только `xs:assert`, `xs:assertion`, `xs:override` и `vc:minVersion` / `vc:maxVersion`.
- XPath: XPath 1.0 полностью; из XPath 2.0 — только подмножество для утверждений XSD 1.1.
- Нет XSLT, DOM Range, особенностей XML 1.1 и загрузки по сети.
- Через Си-интерфейс и обёртки доступны DOM, XPath и XML Schema; собственный SAX-обработчик, `TreeWalker` и свой загрузчик ресурсов — только из C++.
- В WebAssembly нет файловой системы: работа идёт со строками и байтами.

Полный список — в файле `docs/comparison.md` репозитория (раздел «Ограничения»).